

Neyman Orthogonality for the Partially Linear Regression Model

PLSC 40601

2024-04-25

Example

$$Y = D\theta_0 + g_0(X) + U, \quad \mathbb{E}[U|X, D] = 0,$$

$$D = m_0(X) + V, \quad \mathbb{E}[V|X] = 0$$

- Y is the dependent variable.
- θ_0 is the causal parameter of interest.
- D is the binary treatment assignment.
- X is the covariate matrix.
- $g(X)$ is a nuisance function describing how X affects Y .
- $m(X)$ is a nuisance function describing how X affects D .

The associated moment score for (1.5) is given by:

$$\psi(W; \theta, g, m) = (Y - g(X) - \theta D) \times (D - m(X))$$

Here “score” means moment score or estimating equation, not necessarily a likelihood score. The naive score uses D in the second factor; the orthogonal score replaces it with the residualized treatment $D - m(X)$. At the true value of the parameter, the expectation of the score is zero. We choose $\check{\theta}$ by solving the empirical moment equation $\mathbb{E}_n[\psi(W; \theta, \hat{g}, \hat{m})] = 0$, i.e., setting the average score across observations equal to zero.

Here, we estimate the nuisance functions $\hat{g}(X)$ and $\hat{m}(X)$ and then plug them into the estimating equation for θ (1.5):

$$\check{\theta} = \frac{\sum_{i=1}^n (Y_i - \hat{g}(X_i))(D_i - \hat{m}(X_i))}{\sum_{i=1}^n D_i(D_i - \hat{m}(X_i))}$$

The empirical expectation of the score is zero, or approximately zero, at the selected value of $\check{\theta}$:

$$\mathbb{E}_n[\psi(W; \theta, g)] = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{g}(X_i) - \check{\theta} D_i) \times (D_i - \hat{m}(X_i)) \approx 0$$

This is true by construction, because $\check{\theta}$ solves the empirical moment equation. It is not, by itself, evidence that the estimator is unbiased or valid.

The Neyman orthogonality condition is given by:

$$\partial_\eta \mathbb{E}[\psi(W; \theta_0, \eta)]|_{\eta=\eta_0} = 0, \quad \eta = (g, m)$$

Neyman orthogonality implies that the score function used to estimate a parameter of interest is insensitive to small perturbations in the nuisance parameter estimates. The “orthogonality” in the term comes from the idea that the partial derivative (or Gateaux derivative in functional spaces) of the expected value of the score function with respect to the nuisance parameter(s) equals zero at the true value(s) of the parameter(s).

This implies that the score function is orthogonal (in the sense of having zero covariance) to perturbations in nuisance parameters.

A useful heuristic is that the partial derivatives of the score with respect to the nuisance functions are:

$$\frac{\partial \psi}{\partial g} = -(D - m(X))$$

$$\frac{\partial \psi}{\partial m} = -(Y - g(X) - \theta D)$$

But because g and m are functions, the formal condition is directional. Let

$$g_r = g_0 + rh_g, \quad m_r = m_0 + rh_m.$$

Define

$$M(r) = E[(Y - D\theta_0 - g_r(X))(D - m_r(X))].$$

At the truth, $Y - D\theta_0 - g_0(X) = U$ and $D - m_0(X) = V$, so

$$M(r) = E[(U - rh_g(X))(V - rh_m(X))].$$

Therefore

$$M'(0) = -E[h_g(X)V] - E[Uh_m(X)].$$

This equals zero for every valid perturbation h_g, h_m because $E[V|X] = 0$ and $E[U|X] = 0$.

Code

```
# Load necessary libraries
library(glmnet) # for lasso

## Loading required package: Matrix

## Loaded glmnet 4.1-8

# Set seed for reproducibility
# set.seed(60637)

# Functions
fit_g <- function(X, resid) cv.glmnet(X, resid, alpha = 1)
fit_m <- function(X, D) glm(paste0('Y ~ ', paste0('X', 1:p, collapse = ' + ')),
                             family = "binomial", data.frame(Y = D, X))

## Simulate data ----
n <- 1e5 # Number of observations
p <- 50 # Number of covariates
nfolds <- 5 # Folds for cross fitting
folds <- sample(rep(1:nfolds, each = n / nfolds))

X <- matrix(rnorm(n * p), ncol = p)
beta_X <- runif(p, -1, 1)
gamma_X <- runif(p, -1, 1)
U <- rnorm(n, sd = 1)

g0 <- X %*% beta_X # True g(X)
m0 <- 1/(1 + exp(-X %*% gamma_X)) # True m(X)
```

```

theta0 <- 0.5 # True theta
D <- rbinom(n, 1, m0)
Y <- D * theta0 + g0 + U

# Estimation ----
g_hat_model <- m_hat_model <- vector("list", nfolds)
theta_naive_est <- theta_neyman_est <- g_hat_est <- m_hat_est <- V_hat_est <- numeric(n)

## Naive estimation ----
# Fit g_hat using iterative methods where we alternate between estimating g_hat and theta_hat
# (we won't use this theta for estimation)
for (k in 1:nfolds) {
  train_indices <- (folds != k)
  test_indices <- (folds == k)

  # Alternating minimization for g_hat
  convergence_threshold <- 1e-5
  max_iter <- 100
  iter <- 1
  changes <- Inf

  # Estimate theta given a starting g_hat
  g_hatX <- mean(Y[train_indices])
  theta_hat <- sum((Y[train_indices] - g_hatX) * D[train_indices]) / sum(D[train_indices]^2)

  while (iter <= max_iter && changes > convergence_threshold) {

    # Calculate residuals
    resid <- Y[train_indices] - D[train_indices] * theta_hat

    # Refit g_hat
    g_hat_model[[k]] <- fit_g(X[train_indices, ], resid)

    # Check convergence
    g_hatX_new <- predict(g_hat_model[[k]], s = "lambda.min",
                        newx = X[train_indices, ])
    changes <- sum((g_hatX - g_hatX_new)^2)
    iter <- iter + 1

    # Estimate theta given g_hat for the next iteration
    g_hatX <- g_hatX_new
    theta_hat <- sum((Y[train_indices] - g_hatX) * D[train_indices]) / sum(D[train_indices]^2)

  }

  # Save g_hat estimates conditional on X using cross-fit models
  g_hat_est[test_indices] <- predict(g_hat_model[[k]],
                                    s = "lambda.min",
                                    newx = X[test_indices, ])

  # Calculate theta estimate for each fold from (1.3)

```

```

theta_naive_est[test_indices] <- {sum((Y[test_indices] - g_hat_est[test_indices]) *
                                     D[test_indices]) / sum(D[test_indices]^2)}
}

# Aside: Check that the pooled naive moment solution gives the same result
# as the no-intercept linear regression coefficient.
theta_naive <- sum((Y - g_hat_est) * D) / sum(D^2)
theta_naive

## [1] 0.4647056
coef(lm(Y ~ g_hat_est - D-1))

##          D
## 0.4647056

# Naive score. The sample mean is zero by construction because theta_naive
# solves the pooled empirical moment equation.
score_naive <- (Y - g_hat_est - theta_naive * D) * D
mean(score_naive)

## [1] 1.537071e-15

# Oracle naive estimator using true g0
sum((Y - g0) * D) / sum(D^2)

## [1] 0.4979904

# Naive score evaluated at the truth
mean((Y - g0 - theta0 * D) * D)

## [1] -0.001004792

# Fold-averaged naive estimate
mean(theta_naive_est)

## [1] 0.4647178

# Pooled naive estimate + standard error
theta_naive

## [1] 0.4647056
J_naive <- -mean(D^2)
se_naive <- sqrt(mean(score_naive^2)) / abs(J_naive) / sqrt(n)
se_naive

## [1] 0.004482738

#* Neyman orthogonal estimation ----

# Fit m_hat in each of the folds
for(k in 1:nfolds){
  train_indices <- (folds != k)
  test_indices <- (folds == k)

  m_hat_model[[k]] <- fit_m(X[train_indices,],
                           D[train_indices])

```

```

# Save m_hat estimates conditional on X using cross-fit models
m_hat_est[test_indices] <- predict(m_hat_model[[k]],
                                   newdata =
                                   data.frame(X[test_indices,]),
                                   type = "response")

# Save V_hat estimates
V_hat_est[test_indices] <- D[test_indices] - m_hat_est[test_indices]

# Get theta estimate for each fold from (1.5).
# This is DML1: solve a separate estimating equation in each fold.
theta_neyman_est[test_indices] <- {
  sum((Y[test_indices] - g_hat_est[test_indices]) *
       V_hat_est[test_indices]) / sum(D[test_indices]*V_hat_est[test_indices])}
}

# DML1 estimate: average the fold-specific theta estimates.
theta_dml1 <- mean(theta_neyman_est)
theta_dml1

```

```
## [1] 0.4840188
```

```

# DML2 estimate: pool the cross-fitted scores and solve one equation.
theta_dml2 <- sum((Y - g_hat_est) * V_hat_est) / sum(D * V_hat_est)
theta_dml2

```

```
## [1] 0.4839262
```

```

# DML2 score. The sample mean is zero by construction because theta_dml2
# solves the pooled empirical moment equation.
score_dml2 <- (Y - g_hat_est - theta_dml2 * D) * V_hat_est
mean(score_dml2)

```

```
## [1] 6.167807e-16
```

```

# Orthogonal score evaluated at the truth
mean((Y - g0 - theta0 * D) * (D - m0))

```

```
## [1] -0.001377255
```

```

# Estimate + standard error for the score used above
J_dml2 <- -mean(D * V_hat_est)
se_dml2 <- sqrt(mean(score_dml2^2)) / abs(J_dml2) / sqrt(n)
se_dml2

```

```
## [1] 0.01057704
```

```

# Robinson-style regression on V_hat. This is closely related, but not
# algebraically identical to the score above in finite samples: it uses
# sum(V_hat_est^2) rather than sum(D * V_hat_est) in the denominator.
summary(lm(Y - g_hat_est ~V_hat_est-1))

```

```

##
## Call:
## lm(formula = Y - g_hat_est ~ V_hat_est - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max

```

```
## -4.1031 -0.4532 0.2303 0.9160 4.8332
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## V_hat_est    0.4835     0.0110  43.93  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.045 on 99999 degrees of freedom
## Multiple R-squared:  0.01894,    Adjusted R-squared:  0.01893
## F-statistic: 1930 on 1 and 99999 DF,  p-value: < 2.2e-16
```